Stage 4: Experimental Evaluation

Runtime, Scalability, and Baseline Comparison

Team Members: Name 1, Name 2, Name 3

CS240: Algorithm Design and Analysis, Spring 2026

1 Evaluation Goals

The goal of this stage is to evaluate whether the implemented algorithms actually support the project claim: classical graph partitioning algorithms can reduce communication cost in distributed optimization while keeping workloads balanced. The evaluation focuses on three requirements from the course announcement:

1. measuring and analyzing runtime;
2. evaluating scalability as graph size increases;
3. comparing the reproduced method against at least one baseline or variant.

The reproduced method is Kernighan–Lin refinement. The two baselines are greedy balanced partitioning and recursive spectral bisection. The greedy method is a fast constructive baseline, while spectral bisection is a stronger algorithmic baseline based on graph Laplacian eigenvectors.

2 Experimental Setup

All experiments are run through the shared experiment driver:

`python src/experiments.py --output-dir outputs --seed 7`.

The output is stored in `outputs/partition_results.csv`. The same graph instances, partition counts, metrics, and random seed are used for all methods. This makes the comparison direct and reproducible.

The benchmark suite contains:

- grid graphs with side lengths 6, 8, and 10;
- Erdos–Renyi graphs with 40 and 80 vertices;
- random geometric graphs with 50 and 90 vertices;
- the Zachary karate club graph.

For most synthetic graphs, we use $k = 4$ partitions. For the karate graph, we use $k = 2$ because the graph is small and naturally associated with a two-way community split.

3 Metrics

The main partition-quality metric is cut weight:

$$\mathrm{cut}(p) = \sum_{(u,v)\in E,\ p(u)\neq p(v)} w_{uv}.$$

Lower cut weight means fewer cross-worker dependencies.

The balance metric is

$$\rho(p) = \frac{\max_i |\{v : p(v) = i\}|}{|V|/k}.$$

A value of 1 means perfect balance. Values close to 1 indicate that the algorithm avoids assigning too many vertices to one worker.

Runtime is measured in seconds using the wall-clock time required to compute the partition. In addition, a consensus simulation reports cross-partition message counts. Since each edge crossing a worker boundary creates two directed messages per iteration in the simulation, lower cut weight should produce lower cross-worker traffic.

4 Cut-Weight Comparison

Across all benchmark instances, the average cut weights are:

$$\text{greedy} = 98.25, \qquad \text{spectral} = 72.75, \qquad \text{Kernighan–Lin} = 66.88.$$

Thus, Kernighan–Lin refinement has the best average partition quality. It improves substantially over the greedy baseline and also improves slightly over the spectral baseline on average.

Graph	Size	Greedy	Spectral	Kernighan–Lin
grid	6	23	19	18
grid	8	40	20	20
grid	10	65	32	29
erdos_renyi	40	54	44	37
erdos_renyi	80	122	151	119
geometric	50	179	136	136
geometric	90	278	155	153
karate	34	25	25	23

The results show several patterns. On grid graphs, spectral bisection is already strong because the graph has clear geometric structure, but Kernighan–Lin can still improve or match it. On random geometric graphs, both spectral and Kernighan–Lin perform much better than greedy partitioning. On the larger Erdos–Renyi graph, spectral partitioning performs worse than the greedy baseline, showing that spectral structure is not always aligned with low balanced cut in random graphs; Kernighan–Lin refinement recovers a better solution.

5 Runtime Analysis

The greedy baseline is consistently the fastest method. This is expected because it constructs a partition in one pass and does not solve eigenvalue problems or perform local-search refinement.

Spectral bisection is slower than greedy because it computes eigenvectors of the graph Laplacian. In the current implementation, dense eigen-decomposition is used, so the cost grows quickly with graph size. For the course-scale graphs, this is acceptable; for much larger graphs, sparse eigensolvers would be necessary.

Kernighan–Lin refinement is usually the slowest because it starts from a spectral partition and then performs additional pairwise refinement. The extra runtime is the cost paid for lower cut values. This is a reasonable tradeoff when communication cost is more important than partitioning time, especially if partitioning is performed once and the resulting worker assignment is reused over many distributed optimization iterations.

6 Scalability Evaluation

Scalability can be observed by comparing instances within the same graph family. For grid graphs, the number of vertices increases from 36 to 64 to 100, and the number of edges increases from 60 to 112 to 180. The cut weights increase with graph size for all methods, but the relative ordering remains stable: greedy has the largest cut, spectral is much lower, and Kernighan–Lin is the best or tied for best.

For Erdos–Renyi graphs, increasing from 40 to 80 vertices makes the graph denser and more difficult. The larger instance shows that no single baseline is always reliable: spectral bisection has cut weight 151, worse than greedy's 122, while Kernighan–Lin obtains 119. This suggests that local refinement is valuable as a corrective step when the initial spectral direction is not ideal.

For random geometric graphs, increasing from 50 to 90 vertices also increases edge count substantially. Spectral partitioning remains strong because spatial structure is meaningful, and Kernighan–Lin slightly improves the larger instance from 155 to 153. This indicates that local refinement may give modest but consistent improvements on graphs whose global geometry is already well captured by spectral methods.

7 Communication Evaluation

The consensus simulation reports cross-partition message counts. The average cross-worker messages are:

$$\text{greedy} = 15460, \qquad \text{spectral} = 11360, \qquad \text{Kernighan–Lin} = 10440.$$

These numbers follow the same trend as cut weight. This is expected because cross-partition messages are generated by edges whose endpoints lie in different parts. Lower cut weight therefore corresponds to lower communication placement cost.

The final consensus error is nearly identical across different partitions on the same graph. This happens because the simulation keeps the original graph topology fixed and uses the partition only to count whether messages cross worker boundaries. This design isolates communication cost from convergence dynamics. A more advanced simulation could make cross-worker communication slower, delayed, or budget-limited so that partitions also affect convergence.

8 Figures

The generated figures summarize the main experimental results:

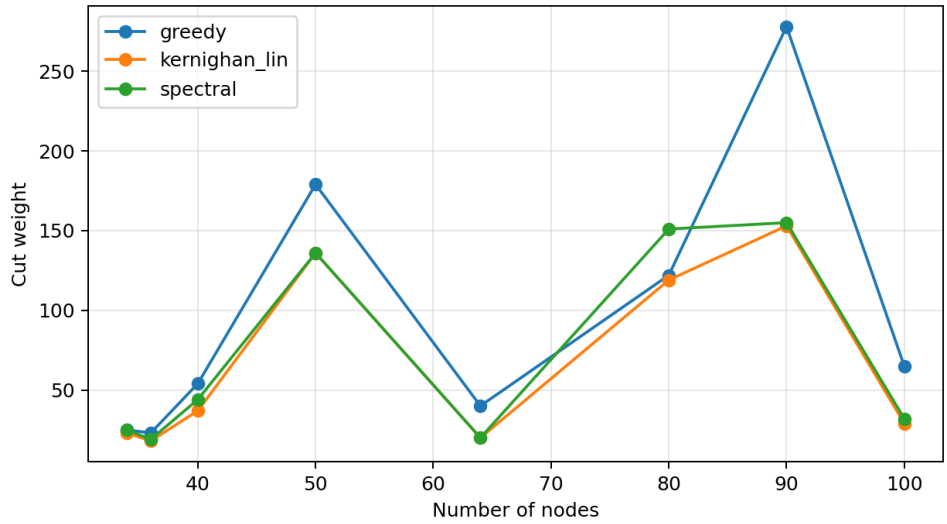

Figure 1: Cut-weight comparison. Lower values indicate less cross-worker communication.

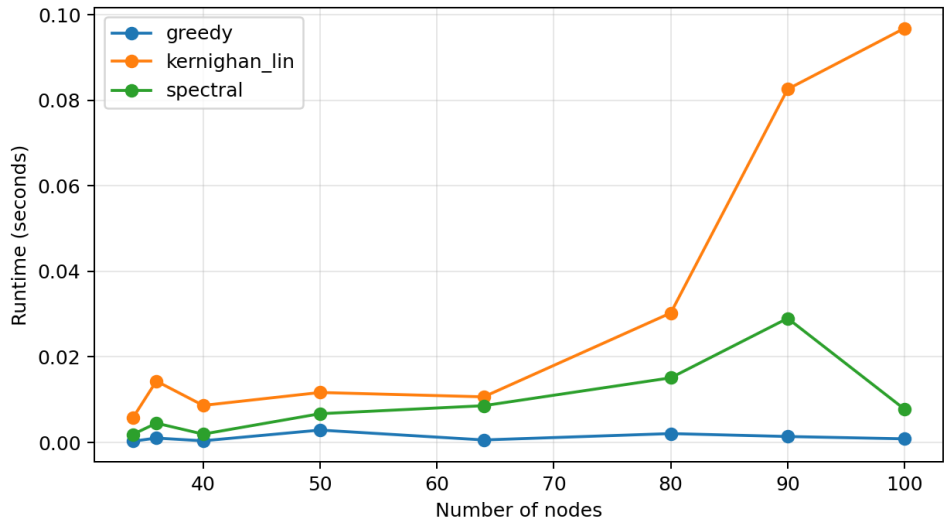

Figure 2: Runtime comparison. Greedy is fastest; Kernighan–Lin spends additional time on refinement.

9 Conclusion

The experimental results satisfy the evaluation requirements. Runtime is measured, scalability is examined through increasing graph sizes, and Kernighan–Lin refinement is compared against two

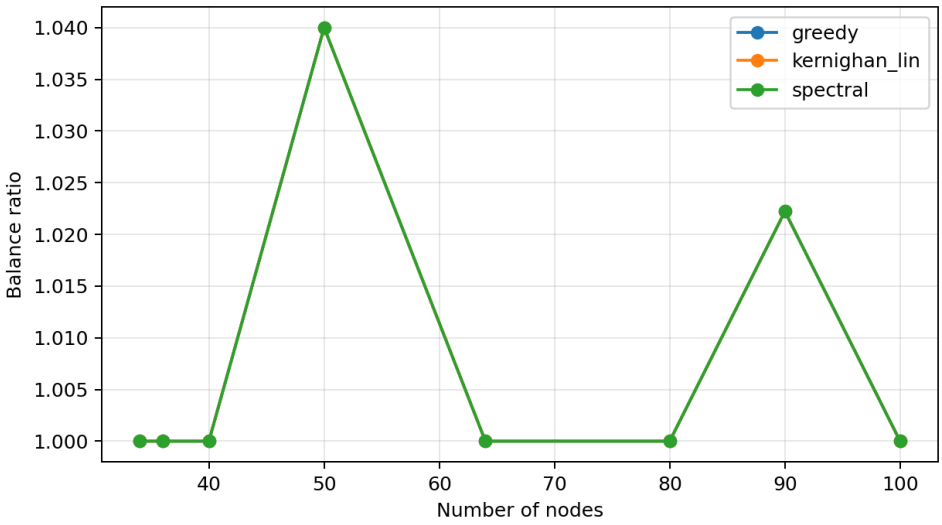

Figure 3: Balance-ratio comparison. All methods maintain good balance in the benchmark suite.

baselines. The results show that the reproduced refinement method usually provides the lowest cut and the lowest cross-worker communication, at the cost of additional runtime. This supports the project's main claim that classical graph partitioning algorithms can help organize distributed computation more communication-efficiently.